

Faculdade de Design,
Tecnologia e Comunicação
 **Universidade Europeia**

Engenharia Informática

Projecto Sesimbra

Afonso Neto	20240910
Alexandre Neves	50032545
André Pereira	20241780
Carlos Pereira	20241813

Rodolfo Agüero Bendoyro

Programação Web

Lisboa, 18, junho, 2026

Links do Projeto

Repositório GitHub	https://github.com/Carlosmsp/Pro-Web
Site intermedio online	https://web-production-1065.up.railway.app
Rep. GitHub de Site intermedio	https://github.com/Carlosmsp/turismo-sesimbra

Enviado convite ao professor deste repositório no início de junho deste repositório que guardou a versão intermédia que estava online .

Índice

1. Introdução.....	3
2. Requisitos do briefing e objetivos do projeto	4
3. Arquitectura do Sistema	5
3.1 Visão Geral.....	5
3.2 Frontend	5
3.3 Backend	6
Endpoints públicos:.....	6
Endpoints administrativos (requerem token):	6
3.4 Persistência.....	7
3.5 Arquitectura Modular	7
3.6 Funcionalidades Adicionais	8
Sugestões de Locais por Utilizadores	8
Recomendações Personalizadas	8
4. Integração de serviços externos.....	9
4.1 Google Gemini (gemini-2.5-flash)	9
4.2 Open-Meteo	9
4.3 OSRM (Project-OSRM)	9
4.4 OpenStreetMap / Nominatim	9
5. Segurança	10
6. Acessibilidade e Usabilidade.....	11
7. Limitações e trabalho futuro	12
8. Conclusão	13
9. Referências.....	14

1. Introdução

O Projecto Sesimbra tem como objectivo o desenvolvimento de um sistema web informativo e interactivo sobre a localidade de Sesimbra. A aplicação integra componentes de frontend e backend, persistência de dados em base de dados relacional e serviços externos de apoio ao utilizador (meteorologia, roteamento e assistente de inteligência artificial).

O projecto visa demonstrar competências técnicas e metodológicas adquiridas nas unidades curriculares de Programação Web (frontend, backend, API, base de dados) e Interfaces e Usabilidade (design centrado no utilizador, acessibilidade, prototipagem e testes).

2. Requisitos do briefing e objetivos do projeto

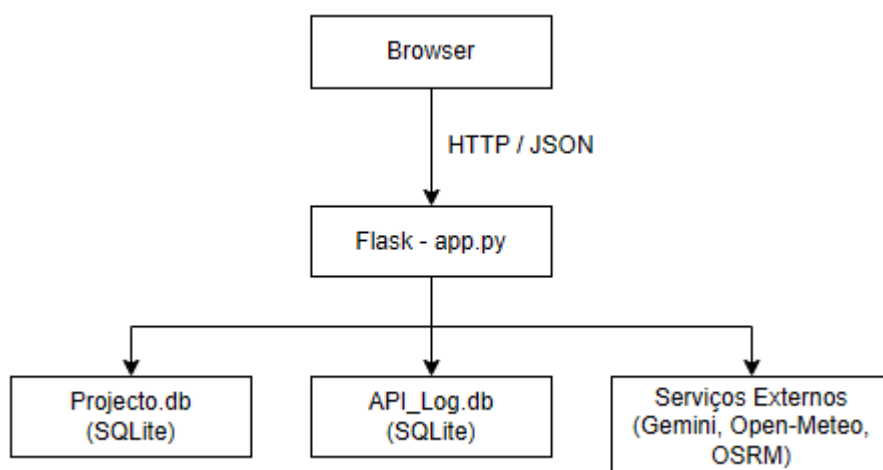
A tabela seguinte mapeia cada requisito obrigatório do briefing para a implementação concreta realizada.

Requisito do briefing	Implementação
Páginas temáticas obrigatórias	frontend/sesimbra.html, frontend/pontos-turisticos.html, frontend/atividades.html, frontend/gastronomia.html, frontend/alojamentos.html, frontend/historia.html, frontend/contactos.html, frontend/admin.html e página 404 personalizada
Pontos de interesse - mínimo 12, pelo menos 3 categorias	API /api/conteudo devolve 9 pontos de interesse (categorias: história, praia, natureza, gastronomia) + 20 restaurantes + 13 alojamentos + 24 actividades (categorias: água, natureza, cultura, aventura), totalizando mais de 66 entradas em 4 categorias distintas
Página de pontos de interesse construída dinamicamente via API própria (modelo SPA)	pontos-turisticos.html implementada como SPA: o HTML é um contentor vazio preenchido em runtime pelo JavaScript via fetch() ao endpoint /api/conteudo, que lê os dados do SQLite
Formulário com validação JavaScript	Formulário de contacto em contactos.html com validação client-side completa (nome, email, telefone, mensagem) e validação server-side espelhada no backend Flask
Processamento server-side com API de Inteligência Artificial	O endpoint /api/chatbot recebe a pergunta do utilizador, constrói um prompt contextualizado com dados da base de dados SQLite e meteorologia em tempo real, e devolve resposta gerada pelo modelo Gemini 2.5 Flash. Esta funcionalidade constitui o formulário interactivo com processamento por IA exigido no briefing.
Princípios de usabilidade, acessibilidade e interactividade	Estrutura HTML semântica, hierarquia de títulos, layout responsivo, dark mode, aria-label, aria-modal, aria-live, navegação por teclado

3. Arquitectura do Sistema

3.1 Visão Geral

O sistema segue a arquitectura cliente–servidor clássica, com separação clara entre frontend e backend. O backend adoptou uma estrutura modular com os módulos `rotas.py`, `seguranca.py`, `configuracoes.py` e `validadores.py`, o que melhora a legibilidade e facilita a manutenção independente de cada componente.



3.2 Frontend

Implementado com HTML semântico, CSS responsivo e JavaScript puro (sem frameworks). O frontend está organizado em páginas HTML na pasta `frontend/` e em vários módulos JavaScript na pasta `assets/js/`, o que separa melhor os dados, a lógica de interface e as funcionalidades específicas. Entre os ficheiros mais relevantes encontram-se `dados.js`, `elementos.js`, `cards.js`, `ui.js`, `imagens.js`, `contactos.js`, `pontos-turisticos.js`, `chatbot.js`, `sugestoes.js`, `recomendacoes.js` e `main.js`.

Os módulos JavaScript são responsáveis por:

- Geração dinâmica da navegação e dos cartões de conteúdo.
- Criação de elementos DOM via `document.createElement`, sem recorrer a `innerHTML` para dados dinâmicos.
- Filtragem em tempo real dos conteúdos apresentados nas páginas.
- Overlay acessível de imagens com suporte a teclado.
- Dark mode com persistência em `localStorage`.
- Formulário de contacto com validação personalizada e submissão via `fetch()` com token CSRF.
- Formulário de sugestões de locais com upload de imagens.
- Assistente IA e secção de recomendações comunitárias.

3.3 Backend

Aplicação Flask (app.py) com arquitectura modular (backend/rotas.py) que expõe os seguintes endpoints REST:

Endpoints públicos:

GET /api/conteudo	Devolve todos os dados do SQLite (pontos, restaurantes, alojamentos, actividades) consumidos pela SPA
POST /api/chatbot	Recebe pergunta do utilizador, constrói prompt com contexto do site e meteorologia, chama o Gemini e regista a interacção em API_Log.db
POST /contactos POST /api/contactos	Valida e persiste contactos do utilizador em Projeto.db, com protecção CSRF e rate limiting por IP
GET /api/csrf	Emite token CSRF para uso nos pedidos POST do frontend.
POST /api/sugestoes	Recebe sugestões de novos locais com imagens, validação dupla e armazenamento em BD
GET /api/recomendacoes	Devolve recomendações personalizadas baseadas no histórico do utilizador

Endpoints administrativos (requerem token):

GET /api/admin/contactos	Devolve lista de contactos
GET /api/admin/contactos/exportar.csv	Exporta contactos em formato CSV.
POST /api/admin/contactos/<id>/resolver	Marca contacto como resolvido.
POST /api/admin/contactos/<id>/apagar	Elimina contacto.
GET /api/admin/sugestoes	Lista sugestões de locais pendentes de moderação.
POST /api/admin/sugestoes/<id>/<acao>	Modera sugestões (publicar/rejeitar).
GET /api/admin/resumo	Devolve resumo estatístico (contactos, sugestões, logs).
GET /api/logs	Devolve histórico das interacções com a IA

3.4 Persistência

Duas bases de dados SQLite independentes, com propósitos distintos:

Base de dados	Tabelas	Conteúdo
Projeto.db	contactos, pontos_interesse, restaurantes, alojamentos, atividades, sugestoes_locais	Dados iniciais populados no arranque da aplicação via INSERT OR REPLACE; sugestões de utilizadores com estado (pendente/publicado/rejeitado) e imagens
API_Log.db	api_logs	Regista cada chamada ao Gemini: endpoint, modelo, prompt, resposta e estado (ok / erro / limitado)

3.5 Arquitectura Modular

Com a evolução do projecto, o código foi refactorizado para uma arquitectura modular que melhora a manutenibilidade, testabilidade e reutilização. O ponto de entrada continua a ser app.py, mas a lógica foi distribuída pelos seguintes módulos:

- backend/rotas.py - definição de todos os endpoints REST da aplicação.
- backend/seguranca.py - validação CSRF, rate limiting, controlo de acesso e configuração CORS.
- backend/configuracoes.py - variáveis globais, carregamento de .env e caminhos do sistema de ficheiros.
- backend/validadores.py - funções de validação de dados (contactos, sugestões, imagens).
- frontend/ - páginas HTML e assets (CSS, JavaScript).
- database/ - scripts auxiliares de base de dados.

Esta separação facilita a adição de testes unitários por módulo e reduz o acoplamento entre lógica de negócio, segurança e configuração.

3.6 Funcionalidades Adicionais

Sugestões de Locais por Utilizadores

A aplicação inclui um formulário de sugestões (POST /api/sugestoes) que permite aos utilizadores propor novos locais para o guia. O fluxo de submissão inclui validação dupla (cliente e servidor) dos campos nome, descrição, categoria e contacto, bem como upload opcional de imagens com validação de tipo (jpg, jpeg, png, webp) e tamanho máximo de 3 MB. Cada sugestão é persistida na tabela sugestoes_locais em Projeto.db com estado de moderação (pendente, aceite, recusada). Os administradores moderam as sugestões via POST /api/admin/sugestoes/<id>/<acao>, usando as ações aceite e recusar.

Recomendações Personalizadas

O endpoint GET /api/recomendacoes devolve apenas conteúdos marcados como origem = 'malta' nas tabelas de pontos de interesse, actividades, gastronomia e alojamentos. A funcionalidade funciona como um canal de recomendação comunitária e não como um motor de personalização baseado em histórico, preferências ou meteorologia.

4. Integração de serviços externos

4.1 Google Gemini (gemini-2.5-flash)

Utilizado no endpoint `/api/chatbot`, para cada pedido, o backend:

1. Lê os dados actuais do SQLite e constrói um contexto dinâmico com pontos, restaurantes, alojamentos e actividades
2. Obtém a meteorologia em tempo real via Open-Meteo
3. Constrói o prompt com o contexto do site, a página actual do utilizador e a sua pergunta
4. Chama a API Gemini com temperatura 0.25 e limite de 900 tokens
5. Detecta respostas cortadas (`finishReason == MAX_TOKENS`) e devolve mensagem adequada ao utilizador

4.2 Open-Meteo

API pública de meteorologia, sem chave de autenticação. Fornece temperatura, precipitação e velocidade do vento para Sesimbra em tempo real. Os dados são incluídos no prompt enviado ao Gemini para contextualizar respostas sobre actividades ao ar livre.

4.3 OSRM (Project-OSRM)

Utilizado para cálculo de rotas quando o chatbot gera roteiros. Suporta os perfis foot (a pé) e driving (carro/autocarro). Se o serviço estiver indisponível, a aplicação recorre a um estimador local baseado em distância em linha recta com factor de correcção 1,35.

4.4 OpenStreetMap / Nominatim

Utilizado para pesquisa auxiliar de contactos de restaurantes. Consulta o nó OSM correspondente ao restaurante e extrai o campo phone do XML da API pública.

5. Segurança

Foram implementadas as seguintes medidas de segurança no backend:

Mecanismo	Descrição
CSRF	Token gerado como cookie (samesite=Strict) e verificado no header X-CSRF-Token via <code>secrets.compare_digest</code> , que evita ataques de timing.
Rate limiting	Em memória (defaultdict de deque): chatbot - máx. 8 pedidos/min e 45/hora por IP, intervalo mín. de 2,5 s; contactos - máx. 5 submissões por IP em 10 min.
CORS	Origens permitidas restritas a <code>http://127.0.0.1:5000</code> e <code>http://localhost:5000</code>
Validação de entrada	Regras de formato, comprimento e regex aplicadas de forma independente no cliente (JS) e no servidor (Python).
Sem innerHTML	Todos os elementos DOM são criados via <code>document.createElement</code> , eliminando vectores de XSS por injeção de HTML.
Variáveis de ambiente	GEMINI API KEY lida de <code>.env</code> ou variável de ambiente.

6. Acessibilidade e Usabilidade

Foram seguidas as boas práticas das normas WCAG e do material da UC Interfaces e Usabilidade:

- Estrutura HTML semântica: header, nav, main, section, article, footer
- Hierarquia de títulos respeitada (h1 > h2 > h3) em todas as páginas
- aria-label em elementos de navegação e controlos interactivos
- aria-modal="true" e gestão de foco no overlay de imagens
- aria-live="polite" para feedback de formulários (erros e confirmações)
- aria-current="page" no link de navegação activo
- Todos os controlos interactivos acessíveis por teclado (Tab, Enter, Escape)
- Contraste e tipografia (Roboto) adequados em modo claro e escuro
- Layout responsivo sem frameworks CSS (media queries em CSS puro)
- Dark mode com preferência persistida em localStorage

7. Limitações e trabalho futuro

Escalabilidade

O SQLite é adequado para o âmbito académico do projecto. Para cenários de produção com múltiplos utilizadores simultâneos, recomenda-se migrar para PostgreSQL ou MySQL e adoptar um servidor WSGI (Gunicorn/uWSGI) com proxy reverso (Nginx).

Autenticação administrativa

O token administrativo actual é estático e configurado por variável de ambiente. Uma evolução natural seria substituí-lo por autenticação baseada em sessões com utilizadores e passwords com hash (ex.: Flask-Login + bcrypt).

Testes automatizados

O projecto não inclui testes unitários nem de integração. Recomenda-se acrescentar testes aos endpoints críticos (/api/chatbot, /contactos) com pytest e Flask test client.

Rate limiting em memória

A implementação actual perde o estado ao reiniciar o servidor. Em produção, deveria ser substituída por uma solução persistente (ex.: Redis com Flask-Limiter).

Internacionalização

O conteúdo está em português europeu. Suportar inglês ou espanhol seria uma evolução natural para um guia turístico.

8. Conclusão

O Projecto Sesimbra constitui uma implementação full-stack funcional que cobre todos os requisitos obrigatórios do briefing: páginas temáticas, SPA com API própria e base de dados SQLite, formulário com validação dupla (cliente e servidor), integração de IA generativa com contexto dinâmico, e princípios de acessibilidade e usabilidade. Para além dos requisitos mínimos, o projecto foi expandido com funcionalidades adicionais - sugestões de locais por utilizadores com moderação, recomendações personalizadas e um painel administrativo completo - e refactorizado para uma arquitectura modular que melhora a manutenibilidade e testabilidade do código.

A separação clara entre frontend e backend, a utilização de JavaScript puro sem frameworks, e a integração de serviços externos reflectem as competências desenvolvidas ao longo das duas unidades curriculares participantes.

9. Referências

Pallets. Flask Documentation. <https://flask.palletsprojects.com/>
Python Software Foundation. Python Documentation. <https://docs.python.org/>
SQLite Consortium. SQLite Documentation. <https://www.sqlite.org/docs.html>
Google. Gemini API Documentation. <https://ai.google.dev/gemini-api/docs>
Open-Meteo. API Documentation. <https://open-meteo.com/en/docs>
Project-OSRM. Routing Engine Documentation. <https://project-osrm.org/>
W3C. Web Content Accessibility Guidelines (WCAG) 2.1. <https://www.w3.org/TR/WCAG21/>
W3Schools. HTML, CSS e JavaScript Reference. <https://www.w3schools.com/>
Frain, B. (2022). Responsive Web Design with HTML5 and CSS, 4th ed. Packt Publishing.
Material de apoio das UCs: Programação Web e Interfaces e Usabilidade. Canvas -
Universidade Europeia, 2025–2026.